UNIVERSITÉ DE LIÈGE

Faculté des Sciences Appliquées



ELEN0037: Microelectronic and IC design VHDL and FPGA : final report

Florentin Godfirnon, Arnaud Duvivier and Alex Rousseau

May 13, 2024

Année Universitaire : 2023-2024

Contents

1 Description of the implementation **2**
- 1.1 Overview of the game . . . 2
- 1.2 Player control . . . 2
- 1.3 Game state management . . . 3
- 1.4 Graphics rendering . . . 3
  - 1.4.1 Basic sprites . . . 3
  - 1.4.2 Sprites background . . . 3
  - 1.4.3 Inversion . . . 3
  - 1.4.4 Walking . . . 4
  - 1.4.5 Jumping . . . 4
  - 1.4.6 Punching . . . 5
  - 1.4.7 Projectiles . . . 5
  - 1.4.8 Upscaling . . . 5
  - 1.4.9 Background . . . 6
- 1.5 Collision management . . . 6
- 1.6 Health and damage system . . . 6
- 1.7 Boosts generation . . . 7
- 1.8 Gravity . . . 7

2 Architecture **8**
- 2.1 Game . . . 8
- 2.2 Description of the entities . . . 8

3 Issues encountered **9**

4 Simulations **10**

5 Final thoughts **11**
- 5.1 Further improvements . . . 11
- 5.2 Conclusion . . . 12

1 Description of the implementation

1.1 Overview of the game

The game is a versus fighting game with two players, akin to Super Smash Bros. Each player has a certain amount of health points, and if it reaches zero, the player loses. The player is able to move to the left, right, jump and go down thanks to gravity. To hurt the opponent, players can punch them, fire projectiles or jump on them. Each player can have two projectiles on screen at the same time, and can fire with a little cooldown. Players also have a dash, which enable to move quickly but also to make some damage. It is also on a cooldown.
The map is composed of the ground and three platforms: one in the middle, surrounded by two others a little bit higher.
Some boosts are also spawning randomly. They can either boost the damage of the punch, the damage of the projectile, the mobility or provide defense.
Once the opponent HP reach 0, the game is won. It can be restarted and played again.



Figure 1: Poke Smash

1.2 Player control

The player control logic is responsible for interpreting input from the SNES controller and translating it into in-game actions for the player characters. This logic forms the basics of the player's interaction with the world, allowing them to move, jump, attack, and shoot projectiles.
The D-pad is used to make the player move. The left arrow is used to go left, the right arrow to the right and if the player is in the air, the bottom arrow can be used to go down faster. This last action can cause damage to the other player. It is also possible to jump using the yellow button "B". The last button to move is the R button, on the top right of the controller. It is used to make a dash.
To hit the opponent at close range, the player can either press "Y", the green button that will make a punch to the left or right according to the direction the player is facing, or the player can press the blue button, "X" to make an attack to the top.
It is also possible to hit the opponent from a long range using the red button that shoot a projectile.
The game can be paused using the "START" button. During that pause, the game can either be restarted using "X" or continued using "START" again.

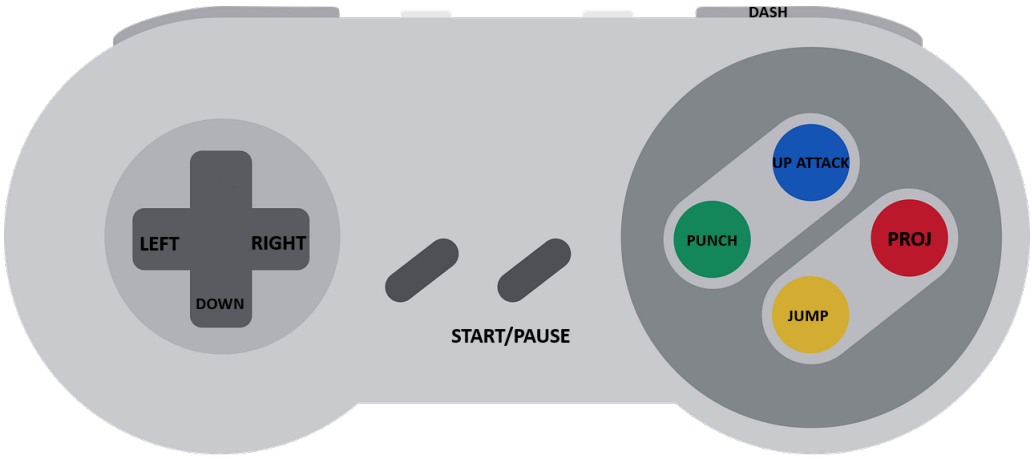


Figure 2: SNES Controller

1.3 Game state management

The game has different state:
At the beginning, the game is in a launch state. Nothing can be done apart from launching the game by pressing the Start button.
When the game is launched, the game entered another state, a play state where the two player can fight until one of them die.
At that time, the game can be paused. It is another state where nothing can be done except restart the game for another match or come back to the current match.
Finally, there are two other states. One when player 1 win and one when player 2 win. Those states let the player that won in the game alone. It can move as much as he wants without any interference with the other player.
All those states are managed by a single entity called "game state" that take as input some information from other entities such as the life of each player for example and then process them in order to update the state of the game.

1.4 Graphics rendering

To save some space, all the designs were pixelated. This meant that it was possible to store them with smaller resolutions. The design were found on the internet or done by ourselves. As the ROM could not handle a "png" format. Those could not be used in the embedded memory. This is why they were converted into "mif" file. It was done using python code transforming images in a list of 12 bit addresses (RGB with 4 bits by color).

1.4.1 Basic sprites

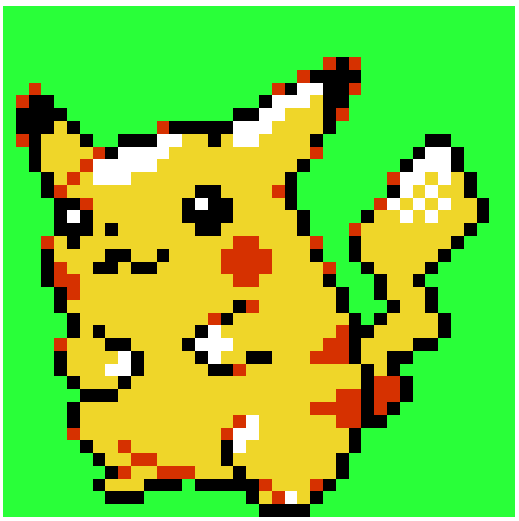
It was decided to use Pokemon sprites for their ease to find, their low pixel count and their undeniable place in the popular culture of our generation. The basic sprites are the following:

1.4.2 Sprites background

To avoid displaying the character'sprite background over platforms, we had to find a way to differentiate the background from the character. We decided to use pure green backgrounds ("000011110000", see figure 3). Those pixels are not to be printed, thus saving the platform color from erasure.

1.4.3 Inversion

To use the less memory possible, we decided to invert the sprites instead of having 2 sprites looking left and right. This is easily implemented by starting to read the last pixel of the first row, then reading backwards as many pixels as the sprite width, and finally jumping at the ROM address two times the width further .



(a) Pikachu sprite

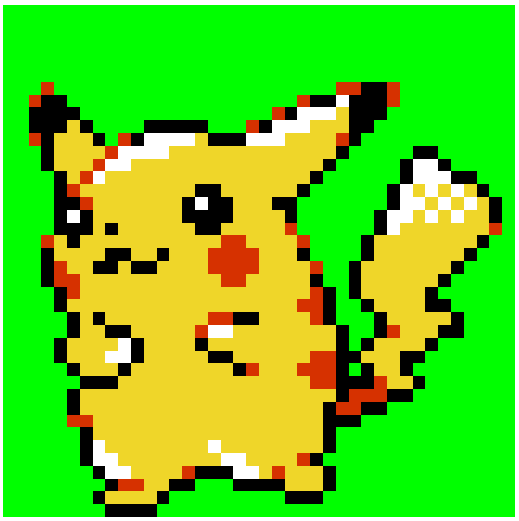


(b) Charmander sprite

Figure 3: Basic sprites

1.4.4 Walking

To simulate walking, the character alternates between two sprites when moving. These two sprites look a little bit different, this is what creates the impression of walking. To do that in VHDL, we used a counter that is launched by the entity "VGA display" when the entity player says that the player begins to walk. This is so the entity player that control the state of the player and the walk that is controlled by the "VGA display".



(a) Pikachu sprite



(b) Charmander sprite

Figure 4: Walking sprites

1.4.5 Jumping

The last sprite possible for our player is the sprite for jumping (or falling, it is the same), this is very simple as a complete sprite is dedicated to this state of the player, the only problem to tackle here is to know when a player is in one state or in another one. The best way to do that is simply to ask the player itself to say in what state it is. And then the vga_display put this sprite at the right place.

1.4.6 Punching

Even if Pokemon sprites are easy to find, Pokemon sprites that punch are very difficult to find. This is why, it was decided to use a sprite of a punch to demonstrate an attack. One sprite is used when punching to the left or to the right, and one for the top attack for each character. That means that four sprites are used. At the beginning, we thought about using just one sprite, but the implementation would have been quite tricky, because if both players punch at the same time, the ROM containing the sprite would have to jump a lot between the addresses. This is hard to implement because of the delay it takes to load the ROM. Moreover, some memory was still available.

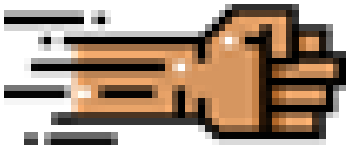


Figure 5: Punch sprite

1.4.7 Projectiles

We also have implemented inversion and "walking" for the projectiles.
Two projectiles of the same player can be on the screen at the same time, meaning they both want to access the same ROM in the same VGA display iteration. This is complicated to implement, as the ROM address must be set some time before use. We have thus decided to force the projectiles to always be in different walking phase, thus using different ROMs to get their sprites.

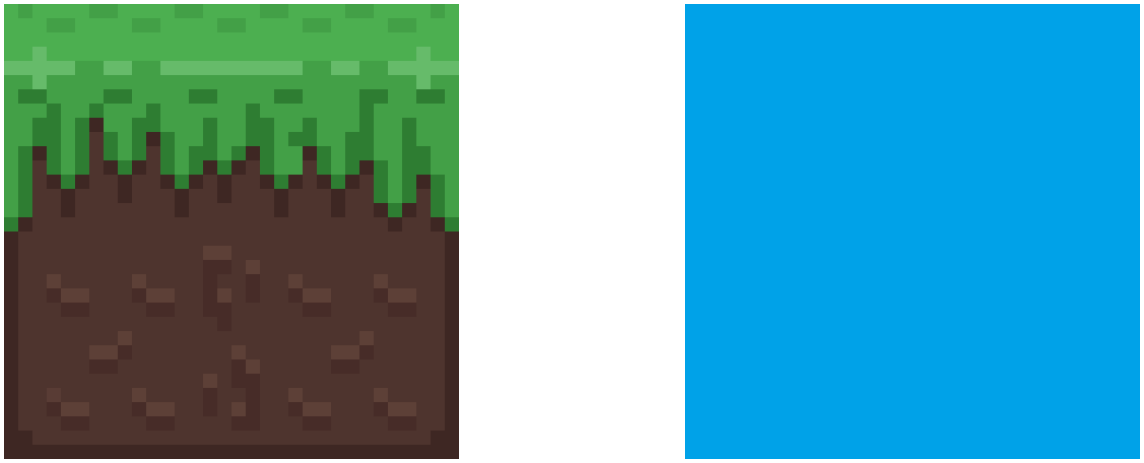
Still, different tricky situations remained : for example in the case where one projectile is fired and the other is not, with projectile 1 using the first sprite and projectile 2 that would use sprite 2. Sprite 2 address has to be set according to the way the character is facing, in case it fires. However when the walking animation happens, projectile 1 accesses the ROM of the second sprite. If the character was looking another way than the projectile 1, that ROM was wrongly set, and has not the time to adjust to the new address needed, thus created a visual bug where a black line would appear.
Indeed, if the ROM address was set to the projectile width, expecting to be used for an inversion, but was instead read incrementally, the ROM will be accessed out of bounds in the end, where values were not initialized and are thus still all zeroes : thus the black colour.
The solution was to preload the ROM according to the way the character is facing only when no projectiles is on screen.
The last unsolved visual bug is when two projectiles are fired in opposite directions : some black lines appear. The problem is the same as described earlier, where the ROM has been preloaded for an inversion but read incrementally. This bug was not solved as this kind of ROM address bug proved to be lengthy to fix, and the bug should appear very rarely. Indeed, having two projectiles facing different directions is hard to achieve because of the size of the map and the speed of the projectiles. With a higher projectile speed, it should even be impossible to happen.

1.4.8 Upscaling

Again, still in the idea to optimize the place used in the ROM to be able to make improvement to the game later, it has been decided to use characters in size 40x40. However, it became quickly obvious that it is too small in order to have a pleasant gameplay. So the best idea in order to have an optimized game and a pleasant gameplay was to double the size of the characters in the display (80x80) but keep the character in the ROM in size 40x40. To do that, some tricky moves had to be done to avoid problems like a tilted character or a black line under a character.

1.4.9 Background

At the top of the screen, there are four different rectangles. The longer ones are the life of each player. The others are the cooldown to be able to do a dash again. On the left, player 1 and on the right, player 2.
Even if we managed to save a lot of memory, it was not really possible to make a complete background as we had wished. This is why the background is composed of a blue color representing the sky with earth blocks.



(a) earth block

(b) background color

Figure 6: Background

1.5 Collision management

This is certainly one of the biggest challenge encountered during this project, more complex than the gravity and the display of the projectiles. The movement of two players moving simultaneously in a same world may seem pretty obvious because the two player must not be able to collide one with the other, but when the gravity and some acceleration are present, this becomes much less trivial. In order to make it more simple, it has been chosen to make the players play sequentially one and then the other. So that each time one player check the position of the other player, he knows that when the player will end all the operations linked to its movement, the other will still be at the same place. Like that, it is useless to take into account the speed of the other player.

1.6 Health and damage system

The health and damage system was pretty easy to implement because it is one of the last feature added to the game and a lot of structures linked to other features have proved to be very useful to implement the health of the player. The check to know if one player is dead is performed by the "game state" entity : it changes the state of the game if the life of one player is equal to zero. The life and the damage linked to one player are managed by the other player. Indeed, the other player knows when it hurts its opponent, because it is the result of a successful attack. Then it has just been useful to check if some conditions are fulfilled when some actions are made by the player to know if his opponent must lose points.

1.7 Boosts generation

The boost generation was the final step of the project, it is the feature where the random of our project take place. The boost is a feature that allow to have an advantage over the opponent on a certain field of the fight. There are four types of boost : a doubling of the projectiles damages, a doubling of the body attack damage, a mobility boost, and a shield. All those boost appear at random moments on the map and the type of boost is also random and depend on the command pressed by the player during the generation of the boost in a pseudo-random way.

The boosts are generated randomly, but the location of the boost is always on a platform or on the ground, but the platform chosen for the apparition is of course chosen randomly as the other characteristics of the boost. To take a boost, the player must walk on it, it will directly appear next to it dash cooldown bar. But this lead to a security that has been taken to avoid some problems, if the boost must appear at a place taken by a player, it will postpone its apparition to another place and at the following clock switch.

In order to ease the display of the boost appearing at different place of the map, we limit the apparition of the boost to only one at a time. Meaning that only one player can have a boost at the same.

1.8 Gravity

The gravity was a complex feature to implement. But this was the first feature implemented in our project, so all the other features were build around this one. This simplified a lot the task to create gravity.

Everything in the gravity works with a variable vertical speed that makes the vertical position vary each tick of the clock. The vertical speed also varies each tick of the clock, which create visually a gravity because the speed increase or decrease depending on the direction of the player. When the player jump, its speed is first set to an initial value, making the player jump, and when it reaches the top of the jump, its speed begin to increase but in the other direction.

All the vertical displacement are completely separated from the horizontal displacement, allowing them to take place at the same time without any disturbances. Only some problems rises when the two players were allowed to collide one with the other because the collision in the corner of the other player were hard to detect, some part of the code in the entity player are only dedicated to tackle this issue.

2 Architecture

2.1 Game

The project is articulated around the entity game, as in the following figure. All the others entities communicate through the "game" entity".

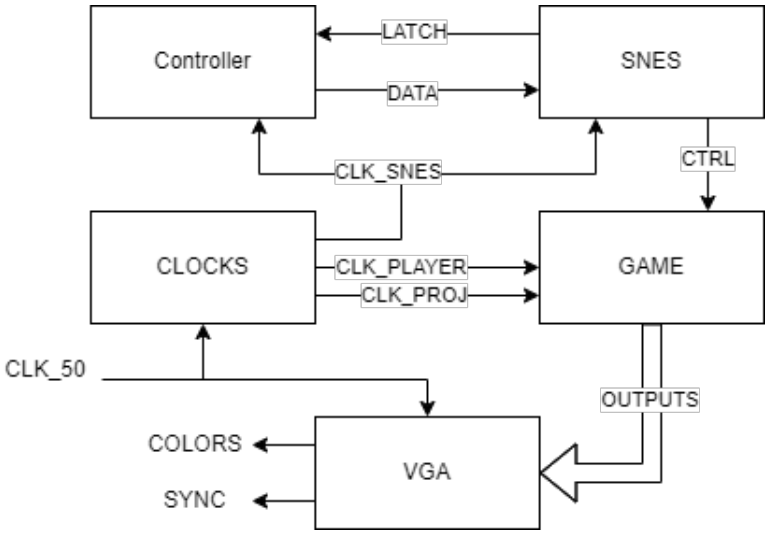


Figure 7: Game diagram

2.2 Description of the entities

1. game : This entity is used to instantiate all the others, except the ROMs. The interactions between the entity game and the environment is visible in figure 7.

2. vga_display : This entity handles the VGA display, using signals coming from game_state, player, projectile and boost. It instantiates the ROM entities needed for the sprites. It also manages all the sprite inversion, the upscaling and the positioning.
   For the display, all the variables that move continuously are only updated when the vertical counter is reset in order to always have a display that is continuous.

3. game_state : This entity determines the state of the game. The different states are listed below:

   - game_state = 0: Launching state. When the FPGA is turned on.
   - game_state = 1: Playing state. Major state of the game. During that state, both players can move and interact with each other.
   - game_state = 2: Pause state. It is the pause menu that is launch when one of the player press start during the game. During that state, none can move or attack. Moreover, the projectiles are stopped in their way.
   - game_state = 3: Player 1 win. He can continue to move the character, but he is alone.
   - game_state = 4: Player 2 win. He can continue to move the character, but he is alone.
   - game_state = 5: Relaunching state. At the end of a game, when the players want to play again, they have to press the START button twice. This state is reached after pressing the START button once. It acts as a stop for the actual game.
   - game_state = 6: Restart the game state. During state 2, if a player presses the X button, the game entered this state. It is a very short state that restart the game.

   It uses signals from snes and player.

4. player : This entity determines the physics of a player. All the signals as his position, his vertical or horizontal speed, his health, whether he is punching, dashing or shooting are in this entity. The entity also takes care about the collision with the environment.
   For each player, an entity must be instantiated.

5. snes : This entity recovers data from the SNES controller.
   Same as for the point before, for each an entity must be instantiated.
   In the entity SNES, the 4-bit integer deliberately lacks a condition to prevent overflow. This intentional choice removes the necessity for an equality check on the 4 bits. However, it causes the counter to cycle through all 16 values. Additionally, the latch is triggered one cycle before overflow to synchronize the counter value ("cnt") with the desired control input ("CONTROL(cnt)").

6. boost : This entity handles the generation of a boost. The choice of the boost as his position is a random process. The boost can therefore be at any position touching the ground or a platform. There are four different boosts: jump boost, punch boost, proj boost and def boost. It uses signals from game_state, player and SNES.

7. clocks : This entity takes as input the standard 50 megaHertz clock and use it to create two slower clocks of 80 kiloHertz and 73 Hertz, named CLK_SNES and clk_player respectively.
   Only vga_display uses the 50 Hertz clock.
   game_state, player, boost and projectile use clk_player.
   snes uses CLK_SNES.
   In most of the entities, when it is possible, the project tries to make instructions in opposite moments of time (opposite tick of the clock), for example, the player instructions operate at the rising edge of clk_player and boost instructions are executed on the falling edge of clk_player.

8. projectile : This entity handles the physics of the two projectiles of a player : the positions, directions,...
   There is one entity per player.

3 Issues encountered

When undertaking this project, we confronted numerous challenges at various stages and on diverse topics, ranging from handling player collisions to accurately displaying characters using ROM.

Regarding the display, the main issues revolved around unwanted black lines beneath a display and images appearing inclined due to incorrect addressing of bits within a specific .vhd file of the ROM memory. The main cause of those visual bugs was the inversion of the sprite, as explained earlier. In hindsight, simply using one sprite for left and one for right would have been far easier, and we would not have run out of memory.
The solutions we found to tackle these problems often involved adjusting address values, as even a difference of just '1' could lead to issues, and careful preloading of the addresses before use of the ROM. Additionally, correcting addressing errors proved effective in resolving display-related issues. Despite the ease with which display issues could arise, they were generally straightforward to resolve.

The problems associated with the development of the game itself were, on the other hand, more complex to solve. Addressing these issues often led to unexpected complications, as resolving one problem sometimes introduced another. Unlike display-related issues, which typically involved straightforward adjustments, game development challenges required more nuanced solutions. For example, implementing collision detection between players appeared straightforward initially. However, upon closer examination, it became evident that certain variables were necessary to address specific aspects of the problem, as signals alone were insufficient for the required operations. Despite numerous attempts using signal-based solutions, a satisfactory resolution remained elusive.

Another significant challenge we encountered was the logistics of project development, particularly with three team members sharing a single FPGA. Collaboration became inherently difficult as we attempted to work simultaneously on different aspects of the project. To alleviate this issue, we acquired a second FPGA. However, the question remained: how could we efficiently divide the workload between two FPGAs? Ultimately, we decided to split the project into two main components: game creation and display management. Initially, the game comprised simple battles between colored squares, allowing us to focus on refining the gameplay mechanics before integrating character sprites to enhance the overall gaming experience.

But speaking to other people making the project, we realized that our problems were minor and easy problems, we did not have problems of speed for the display or time constraints that were not respected.

4 Simulations

The main entity of the project is the entity "Player". It is the most important one as it manages all the movements and interaction of the player. This is why the simulation of the player was made.

As discussed during the VHDL laboratory sessions, due to the numerous external sources of information requiring player intervention, replicating complex behaviors in simulation can be challenging. However, basic functionalities of the entity can still be assessed to ensure proper functionality.

For instance, one crucial aspect of the player is its operation frequency, which should occur only once every two clock ticks. This can be easily verified in simulation, as demonstrated in the following figure, where the values are updated every two clock ticks:

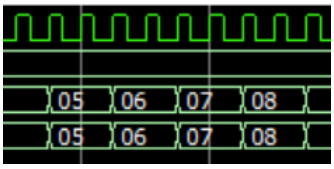
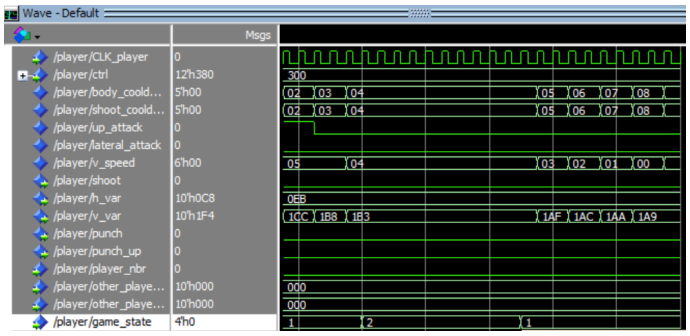
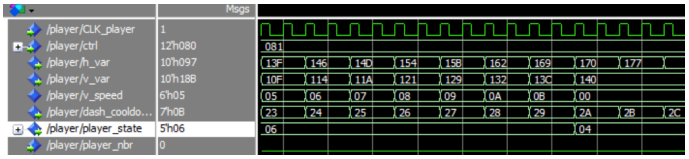


Figure 8: skip of one clock tick

Another significant aspect to consider is the effect of changes in the game_state and the player's state based on the current player actions. Typically, the player should halt any ongoing operations if the game_state deviates from the value 1. Regarding the player's state, which is an output of the entity, updates should occur when the player jumps or falls onto a platform. These functionalities are depicted in the following figures.



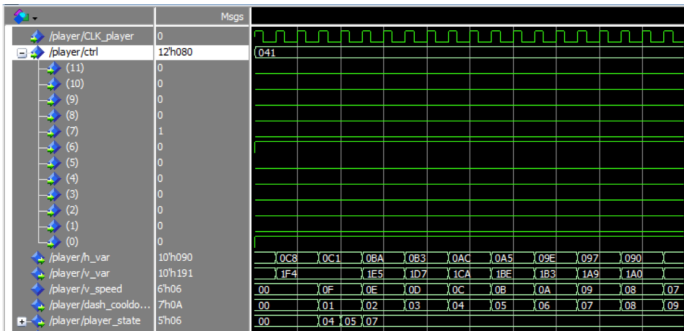
(a) impact of a change of the state of the game



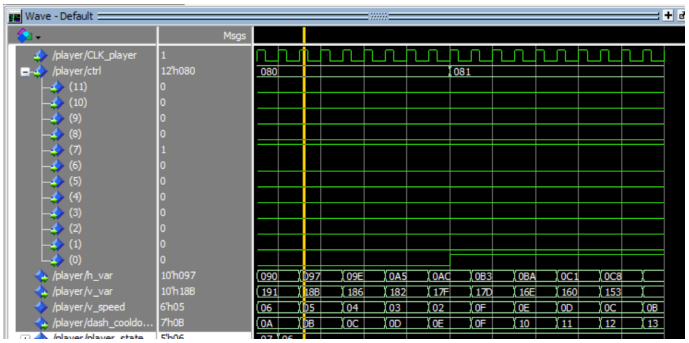
(b) change of the state of the player

Figure 9: states changes

It's also valuable to examine the character's movement when prompted by external player inputs, such as jumping or moving left or right. The character's behavior can be observed by analyzing the signals v_var and h_var, which respectively denote the character's vertical and horizontal positions. In the accompanying figures, the player initiates a jump and directional movement, followed by a change in direction indicated by a different control signal set to '1'. Subsequently, the jump command is deactivated and then reactivated to execute a double jump. At the simulation's conclusion, it becomes evident that the character's vertical position is lower than its initial state, indicative of the character landing on a platform.



(a) first part of the displacement



(b) second part of the displacement

Figure 10: displacement left, right and jump

The final simulation that will be developed in this report concern the body attack and the shoot of a projectile. First, to do this simulation, it was needed to wait until the cooldown of both the projectile and of the shoot reaches their maximal value. Then, the button to shoot and attack (upward in this simulation) are put to one. Then the waited update can be observed, the shoot output tells that the player has shoot. For the attack, it can be seen that the v_var is changed because a vertical attack lead the player to make a little jump (the signal v_speed also change to allow this jump).

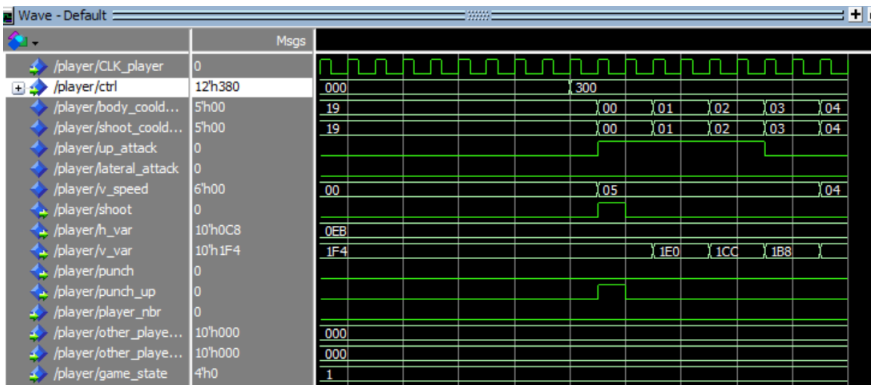


Figure 11: Upward attack and shoot

5 Final thoughts

5.1 Further improvements

As further improvements, many ideas came to us. Some of them are listed below:

- Make more than one map.
- Make a complex background with which it would be possible to interact.
- Personalize each character, so the gameplay would be a little different for the player.
- Add the score of each player at the top of the screen.

5.2 Conclusion

In conclusion, the development of our VHDL-based Super Smash Bros-like game has been a challenging yet rewarding journey. Throughout the process, we encountered various technical hurdles, from implementing player controls to managing collisions and rendering graphics efficiently. Despite these challenges, we successfully created a functional game that captures the essence of the classic game fighting genre.